

6

Building Automated Security Pipelines with Python Using Third-Party Tools

In the previous chapter, we talked about cloud security, data extraction, and exploitation. This part is all about making it easier. This chapter looks into how Python's different libraries and tools can be used to create efficient automated security pipelines. By incorporating third-party tools, we may improve the functionality and scope of these pipelines, assuring comprehensive protection and efficient security operations. We'll discuss being proactive, which includes anticipating future problems. In doing so, we'll use Python, and we'll combine it with additional tools to automate jobs.

In this chapter, we're going to cover the following main topics:

- The art of security automation – fundamentals and benefits
- What is an application programming interface (API)?
- Designing end-to-end security pipelines with Python
- Integrating third-party tools for enhanced functionality
- Ensuring reliability and resilience in automated workflows
- Monitoring and continuously improving security pipelines

The art of security automation – fundamentals and benefits

Cybersecurity automation is a method of automating security tasks to reduce the time and effort required to respond to threats. This approach leverages advanced technologies to detect, prevent, contain, and recover from cyber threats with greater efficiency and precision. By automating repetitive and time-consuming security tasks, organizations can focus on more strategic initiatives and respond to incidents in real time.

Automation in cybersecurity not only improves the speed and accuracy of threat detection and response but also helps in managing the growing complexity and volume of security threats.

The benefits of cybersecurity automation

Automating cybersecurity processes offers numerous advantages, especially in busy environments with high workloads. Here are the key benefits:

- **Enhanced efficiency:** Automation streamlines tasks in cybersecurity departments, reducing the need for manual intervention. This efficiency boost allows professionals to allocate time to more critical areas, minimizing the workload and associated costs.
- **Proactive cyber threat defense:** Automated systems can detect and thwart potential cyberattacks in real time, preventing escalation. Continuous network monitoring provides a robust defense against unauthorized access and safeguards sensitive data.
- **Error reduction:** Human error is a common risk in cybersecurity. Automation eliminates the potential for mistakes such as forgetting password updates or neglecting software upgrades, enhancing overall system reliability.
- **Threat intelligence and analysis:** Automated cybersecurity systems provide rapid identification of emerging threats. By storing detailed activity logs, these systems offer valuable insights into attack patterns, enabling proactive measures to fortify data security.

In summary, cybersecurity automation not only improves operational efficiency but also fortifies defenses, reduces errors, and empowers businesses with actionable threat intelligence.

Functions of cybersecurity automation

Cybersecurity automation streamlines various business operations through the following functions:

- **Detection and prevention:** One of the primary roles of cybersecurity automation is fortifying business defenses against potential threats. It swiftly identifies risks and employs automated solutions to halt further damage. While automation is crucial, a comprehensive strategy may also involve integrating specific tools such as **Residential Proxies** for enhanced protection in areas such as **IP Masking**, **Malware Defense**, **Email Filtering**, **Web Application Firewalls (WAFs)**, and **Intrusion Detection Systems (IDSs)**.
- **Forensics and incident response:** Automation, particularly powered by AI, plays a crucial role in forensics, gathering evidence to understand system breaches. Incident response involves reacting effectively to such incidents and ensuring a well-prepared plan for network attacks. Automated systems aid in comprehending the extent of breaches and guide teams on the necessary steps during and after an attack.
- **Remediation:** Automated remediation accelerates problem resolution. In the aftermath of an attack, manual tasks can be time-consuming and error-prone. Automated remediation allows IT teams to swiftly address issues, enabling a faster return to normal operations. It ensures accuracy and efficiency in each step, preventing repeated mistakes through automatic detection and immediate alerts if anything goes awry during tasks such as software patching or updates.
- **Compliance:** Cybersecurity automation serves as a robust tool for enforcing security policies and procedures, demonstrating a commitment to information security compliance. In regulated industries such as healthcare or finance, automation becomes essential for showcas-

ing due diligence and adherence to best practices. It provides a proactive approach to security, highlighting a dedication to maintaining a secure network and potentially reducing accountability concerns.

Incorporating cybersecurity automation into your strategy not only enhances overall security but also contributes to operational efficiency and regulatory compliance.

Cybersecurity automation best practices

Implementing cybersecurity automation requires adherence to best practices to effectively scale your security efforts and adapt to the dynamic cyber threat landscape. Here are some key guidelines to keep you on the right track:

- **Establish a comprehensive security automation plan:** Develop a clear plan for integrating automation into your cybersecurity strategy and adhere to it consistently.
- **Regularly test automated processes:** Conduct routine tests to ensure that automated processes are functioning as intended and are capable of responding effectively to emerging threats.
- **Evaluate the benefits and drawbacks of automation:** Consider the advantages of automation in enhancing security and also assess potential drawbacks that may arise if automation is not utilized appropriately.
- **Phased implementation:** Roll out automation gradually, starting with addressing commonly occurring security threats. This phased approach allows for smoother integration and adaptation.
- **Integration with existing systems:** Integrate automation seamlessly with your existing systems to create a cohesive and efficient cybersecurity infrastructure.
- **Centralized data storage:** Utilize a centralized database for critical data storage. This facilitates quick identification of issues and enables prompt resolution.
- **Engage third-party service providers:** Consider outsourcing cybersecurity processes to a reputable third-party service provider. This can relieve your organization of the technical complexities associated with maintaining an effective cyber defense program.
- **Employee training:** Educate your staff, particularly the security team, on effectively utilizing automated cybersecurity systems. Clearly define the roles of both humans and machines in the cybersecurity framework.

In conclusion, embrace the power of cybersecurity automation to enhance your organization's security posture. Detect threats early, prevent attacks, and minimize damage with this powerful tool.

Before getting straight into automation, you should be familiar with APIs. Understanding APIs is essential because they form the backbone of automated workflows, facilitating data exchange, triggering automated actions, and enhancing the overall efficiency of security operations.

What is an API?

An API is essentially a contract between two software applications. It specifies how software components should interact, what data they can request, and what actions they can perform. APIs enable the integration of different software systems, allowing them to work together seamlessly. APIs let developers use certain features or get data from a service without needing to know how that service works inside.

APIs have the following components:

- **Endpoints:** Specific URLs or URIs that an API exposes for different functionalities.
- **Request methods:** HTTP methods such as **GET**, **POST**, **PUT**, **DELETE**, and others. These are used to perform different actions on the resources.
- **Request and response formats:** APIs define how data should be structured when it's sent to the API (request) and how the API will structure its response.

Let's imagine an API for a book catalog, and discuss the aforementioned components for it. In this API, we might have different endpoints representing various functionalities:

- **/books:** This endpoint could be used to retrieve a list of all the books in the catalog.
- **/books/{id}:** This endpoint could be used to retrieve details about a specific book, where **{id}** is the unique identifier of the book.

So, the API might expose the following URLs:

- **https://api.example.com/books**
- **https://api.example.com/books/123** (assuming **123** is the ID of a specific book)

Now, coming to request methods, HTTP methods such as **GET**, **POST**, **PUT**, and **DELETE** are used to perform different actions on the resources represented by the endpoints. Let's look at some examples:

- **GET /books:** Retrieves a list of all books
- **GET /books/123:** Retrieves details about the book with ID **123**
- **POST /books:** Adds a new book to the catalog
- **PUT /books/123:** Updates the details of the book with ID **123**
- **DELETE /books/123:** Deletes the book with ID **123** from the catalog

As for the request and response formats, APIs define how data should be structured when it's sent to the API (request) and how the API will structure its response.

For example, when adding a new book (**POST** request), the request might be in JSON format, specifying details such as title, author, and genre. The API might expect a request similar to the following:

```
{ "title": "The Great Gatsby", "author": "F. Scott Fitzgerald", "genre": "Fiction" }
```

In response to a **GET** request for a specific book, the API might return information in a structured format, such as JSON:

```
{ "id": 123, "title": "The Great Gatsby", "author": "F. Scott Fitzgerald", "genre": "Fiction" }
```

In summary, the combination of endpoints, request methods, and request/response formats allows developers to interact with an API in a standardized way. It provides a clear and consistent means of accessing and manipulating data in the book catalog, or any other system for which the API is designed.

With this fundamental understanding of APIs, we can move on to the next section, where we will cover the design and development of security pipelines.

Designing end-to-end security pipelines with Python

A **security pipeline** can be envisioned as a strategic assembly line of automated processes and tools that are designed to fortify applications against potential threats and vulnerabilities. It extends beyond the traditional boundaries of development, reaching into deployment and operational phases. The essence lies in integrating security seamlessly into the software development lifecycle, embodying the principles of **DevSecOps**.

The significance of a security pipeline in the context of cybersecurity can be outlined as follows:

- **Early detection of vulnerabilities:** By integrating security checks into the development process, vulnerabilities can be identified early in the lifecycle, reducing the cost and effort required to fix them. This proactive approach is crucial in preventing security issues from reaching production.
- **Consistent security practices:** Security pipelines enforce consistent security practices across the development, deployment, and operation phases. This consistency helps in maintaining a robust security posture and reduces the risk of overlooking security measures.
- **Automation of security processes:** Security pipelines automate various security processes, such as code analysis, vulnerability scanning, and compliance checks. Automation not only accelerates the development pipeline but also ensures that security measures are consistently applied without relying solely on manual efforts.
- **Continuous monitoring and improvement:** A security pipeline facilitates continuous monitoring of applications and systems for security issues. This continuous feedback loop allows teams to adapt to evolving threats, update security controls, and improve the overall security posture over time.
- **Integration with DevOps practices:** Security pipelines align with DevOps principles by seamlessly integrating security into the **continuous integration/continuous deployment (CI/CD)** workflows. This in-

tegration ensures that security is not a bottleneck but rather an integral part of the rapid and iterative development process.

An end-to-end security pipeline covers the entire software development lifecycle, from the initial stages of code development to deployment, as well as ongoing operations. It involves the following key stages:

1. **Development phase:** Security checks begin in the development phase, where secure coding practices are enforced. Developers utilize tools for static code analysis, identifying and addressing security vulnerabilities in the early stages of writing code.
2. **Build and integration phase:** During the build and integration phase, the security pipeline performs automated tests, including **Dynamic Application Security Testing (DAST)**, **Dependency Scanning**, and other security checks. This ensures that the built artifacts are free from vulnerabilities before the deployment stage takes place.
3. **Deployment phase:** Security controls are applied as part of the deployment process, ensuring that the application is configured securely and that no new vulnerabilities are introduced during deployment. Container security checks may also be included if the application is containerized.
4. **Operations and monitoring phase:** Continuous monitoring is a key component of an end-to-end security pipeline. Security measures such as log analysis, intrusion detection, and anomaly detection help with identifying and responding to security incidents promptly.
5. **Feedback loop and iterative improvement:** The security pipeline provides a feedback loop that allows teams to continuously improve security measures. Lessons learned from security incidents or vulnerabilities discovered in production are fed back into the development cycle, fostering a culture of continuous improvement.

In summary, an end-to-end security pipeline is a comprehensive approach to integrating security into every phase of the software development lifecycle. It ensures that security is not a one-time consideration but a continuous and integral part of the development and operational processes, contributing to a more resilient and secure application or system.

Even though the use of Python in creating a DevSecOps pipeline is minimal, we can always use Python to write intermediate scripts for various purposes.

To build upon this foundation, next, we'll explore how to integrate third-party tools to enhance the functionality and effectiveness of our security pipeline.

Integrating third-party tools for enhanced functionality

This section covers the process of using Python so that you can include the popular web application security scanner ZAP in your security workflow. You can speed up vulnerability assessments and easily incorporate them into your development cycle by automating ZAP scans. We chose

ZAP because it is the most widely used web application scanner on the market, is open source, and is extremely powerful. Additionally, we'll explore how to leverage CI/CD for automation and how to integrate Beagle Security, a proprietary automated penetration testing tool for web applications and APIs.

ZAP is a widely used open source web application security scanner. It helps in identifying security vulnerabilities in web applications during the development and testing phases. ZAP provides a range of features, including automated scanning, passive scanning, active scanning, and API access, making it an excellent tool for integrating into automated security pipelines.

Why automate ZAP with Python?

Automating ZAP using Python offers several advantages:

- **Efficiency:** Automation reduces the manual effort required for security testing, allowing teams to focus on other critical tasks.
- **Consistency:** Automated tests ensure that security scans are performed consistently across different environments and iterations.
- **Integration:** Python's extensive libraries and frameworks make it easy to integrate ZAP into existing CI/CD pipelines and toolchains.
- **Customization:** Python allows you to easily customize ZAP scans to fit specific project requirements.
- **Scalability:** Automated scans can be easily scaled to accommodate large and complex web applications.

Setting up the ZAP automation environment

Before we dive into automating ZAP with Python, let's set up our environment:

1. **Install ZAP:** Download and install ZAP from the official website (<https://www.zaproxy.org/>).
2. **Python environment:** Ensure you have Python installed on your system. You can download Python from <https://www.python.org/> and set up a virtual environment for your project.
3. **ZAP API key:** Generate an API key in ZAP. This key will be used to authenticate API requests from our Python scripts.

Automating ZAP with Python

Now, let's look into the process of automating ZAP using Python:

1. **Install the required Python packages:** We'll need the `python-owasp-zap-v2` package to interact with ZAP programmatically. Install it using `pip`:

```
pip install python-owasp-zap-v2
```
2. **Initialize a ZAP session:** In our Python script, we'll start by initializing a session with ZAP:

```
from zapv2 import ZAPv2
zap = ZAPv2()
```

3. **Configure target URLs:** Specify the URLs of the web applications you want to scan:

```
target_url = 'http://example.com'
```

4. **Perform an active scan:** Next, we'll trigger an active scan on the specified target URL:

```
scan_id = zap.spider.scan(target_url)
zap.spider.wait_for_complete(scan_id)
scan_id = zap.ascan.scan(target_url)
zap.ascan.wait_for_complete(scan_id)
```

5. **Get scan results:** Once the scan is complete, we can retrieve the scan results:

```
alerts = zap.core.alerts()
for alert in alerts:
    print('Alert: {}'.format(alert))
```

6. **Generate a report:** Finally, we can generate a report of the scan's findings:

```
report = zap.core.htmlreport()
with open('report.html', 'w') as f:
    f.write(report)
```

Let's enhance the provided script by adding a function to send the result to a webhook. This will allow us to integrate it seamlessly with communication platforms such as Slack or Microsoft Teams, which typically require specific formats to accept and display the results effectively. You can format the result however you see fit. So, let's add that function:

```
import requests
from zapv2 import ZAPv2
def send_webhook_notification(report):
    webhook_url = 'https://your.webhook.endpoint' # Replace this with your actual webhook URL
    headers = {'Content-Type': 'application/json'}
    data = {'report': report}
    try:
        response = requests.post(webhook_url, json=data, headers=headers)
        response.raise_for_status()
        print("Webhook notification sent successfully.")
    except requests.exceptions.RequestException as e:
        print(f"Failed to send webhook notification: {e}")
def main():
    # Step 2: Initialize OWASP ZAP Session
    zap = ZAPv2()
    # Step 3: Configure Target URLs
    target_url = 'http://example.com'
    # Step 4: Perform Active Scan
    scan_id = zap.spider.scan(target_url)
    zap.spider.wait_for_complete(scan_id)
    scan_id = zap.ascan.scan(target_url)
    zap.ascan.wait_for_complete(scan_id)
    # Step 5: Get Scan Results
    alerts = zap.core.alerts()
    for alert in alerts:
```



```

        print('Alert: {}'.format(alert))
    # Step 6: Generate Report
    report = zap.core.htmlreport()
    # Step 7: Send Webhook Notification
    send_webhook_notification(report)
    with open('report.html', 'w') as f:
        f.write(report)
if __name__ == "__main__":
    main()

```

In this updated script, I've defined a `send_webhook_notification` function that takes the generated report as input and sends it to the specified webhook URL using an HTTP POST request. The `main` function remains the same, but after generating the report, it calls the `send_webhook_notification` function to send the report to the webhook endpoint.

Note that you should replace `'https://your.webhook.endpoint'` with the actual URL of your webhook endpoint.

With this addition, the script will now send the scan results to the specified webhook endpoint after completing the security scan. Make sure your webhook endpoint can receive and process the incoming data accordingly.

Now, let's explore CI/CD, the method we'll use to integrate ZAP into the development workflow.

CI/CD – what is it and why is it important for security automation?

CI means developers regularly add their code changes to a shared code base. Each time this happens, automated tests are run to catch any mistakes early. **CD** goes a step further, automatically putting those changes into action in the real world after they pass all tests.

Let's take a look at why CI/CD is important for security automation:

- **Faster updates:** CI/CD lets us deliver updates to our software quickly and safely.
- **Better quality:** Automated testing helps us find and fix problems before they affect users.
- **Less manual work:** With automation, we spend less time doing repetitive tasks.
- **Team collaboration:** CI/CD brings developers, testers, and operations teams together to work more efficiently.

Now, let's see how we can use Jenkins to automate ZAP.

Introduction to Jenkins

Jenkins is a free tool that helps set up and manage CI/CD pipelines. It's easy to customize and works with many other tools. Jenkins makes it simple to automate tasks such as building, testing, and deploying software.

Let's understand why we should use Jenkins for security automation:

- **Free and open source:** Jenkins doesn't cost anything to use, and anyone can contribute to its development.
- **Flexible:** Jenkins can be customized to work with different tools and technologies, making it adaptable to different projects.
- **Supportive community:** There's a big community of Jenkins users who share tips and help each other out.
- **Scales easily:** Jenkins can handle projects of all sizes, from small teams to large organizations.

Integrating the ZAP automation script into a Jenkins pipeline involves defining stages and steps in **Jenkinsfile** format to execute the script as part of the pipeline. Let's learn how to set up a Jenkins pipeline to run the ZAP automation script:

1. **Configure Jenkins:** First, ensure that Jenkins is set up and configured correctly on your system.
2. **Create a Jenkins pipeline:** Create a new pipeline project in Jenkins and configure it to use a **Jenkinsfile** file from source control (for example, a Git repository).
3. **Define the stages and steps in a Jenkinsfile:** The following is an example **Jenkinsfile** that defines the stages and steps to execute the ZAP automation script:

```

pipeline {
    agent any
    stages {
        stage('Initialize') {
            steps {
                // Checkout source code from repository if needed
                // For example: git 'https://github.com/your/repository.git'
            }
        }
        stage(' ZAP Scan') {
            steps {
                sh '''
                    python3 -m venv venv
                    source venv/bin/activate
                    pip install python-owasp-zap-v2 requests
                    python owasp_zap_scan.py
                '''
            }
        }
    }
}

```

4. **Script execution:** Here's a breakdown of the execution process to provide context for each substep:
 1. The **agent any** directive tells Jenkins to execute the pipeline on any available agent.
 2. The **stages** block defines the different stages of the pipeline.
 3. The **Initialize** stage checks out the source code from the repository if needed.
 4. The **ZAP Scan** stage executes the ZAP automation script. In this example, it activates a Python virtual environment, installs the re-

quired packages, and executes the script (`zap_scan.py`).

5. Ensure that `zap_scan.py` and the Jenkinsfile are present in the source code repository.
5. **Save and run the pipeline:** Save the `Jenkinsfile` file, configure any additional settings if necessary, and run the pipeline.
6. **View the results:** Once the pipeline execution is complete, you can view the results, including the ZAP scan report and webhook notifications if configured.

IMPORTANT NOTE

Ensure that the Jenkins environment has Python installed and that it has access to the internet to download the required packages.

Customize the pipeline script according to your project's requirements, such as configuring Git repository details, specifying Python versions, and adjusting paths as needed.

Set up webhook endpoints to receive notifications from the pipeline as required.

By following these steps, you can integrate the ZAP automation script into a Jenkins pipeline to automate security testing in your CI/CD workflow.

We have successfully created an automated pipeline with the open source tools ZAP and Jenkins. With a few minor code modifications, you can integrate this into your development cycle as the concept remains the same – just the tools you explicitly require have to be identified.

This time, let's integrate Beagle Security, a proprietary program, into our workflow.

Integrating Beagle Security into our security pipeline

In this section, we'll explore how to automate the process of testing an application using Beagle Security's API and Python. Beagle Security provides a comprehensive suite of APIs that enable developers to seamlessly integrate security testing into their CI/CD pipelines or automation workflows. By leveraging these APIs, developers can initiate tests, monitor their progress, retrieve results, and much more, all done programmatically.

Understanding Beagle Security's API

Before we delve into the automation process, let's familiarize ourselves with the key endpoints provided by Beagle Security's API:

1. **Start a test (POST /test/start):**
 1. Initiates a security test for a specified application
 2. Requires the application token
 3. Returns the status URL, result URL, result token, and a message indicating the success or failure of the test start
2. **Stop a test (POST /test/stop):**

1. Halts a running test
 2. Requires the application token
 3. Returns a status code and a message indicating the success or failure of the stop request
3. **Get test result (GET /test/result):**
1. Retrieves the result of a completed test in JSON format
 2. Requires the application token and result token
 3. Returns the test result in JSON format, along with a status code and a message

To fully utilize the potential of the Beagle Security platform, you can benefit from the versatile and user-friendly design of the v2 APIs. The whole API documentation is available at

<https://beaglesecurity.com/developer/apidoc>; however, we will only be using a selection of them in this chapter.

Now that we have a clear understanding of the API endpoints provided by Beagle Security, let's proceed with automating the testing process using Python.

Automating testing with Python

To automate the testing process, we'll utilize Python's **requests** library to interact with Beagle Security's API endpoints. The following is a step-by-step guide on how to implement each part of the automation process:

1. **Retrieve projects and create new ones:** Before we can begin testing methods, we must check that our project is present within Beagle Security. If it is missing, we will quickly create a replacement:

```
import requests
def get_projects():
    # Retrieve existing projects
    url = "https://api.beaglesecurity.com/rest/v2/projects"
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN"
    }
    response = requests.get(url, headers=headers)
    return response.json()
def create_project(name):
    # Formulate a new project
    url = "https://api.beaglesecurity.com/rest/v2/projects"
    headers = {
        "Content-Type": "application/json",
        "Authorization": "Bearer YOUR_ACCESS_TOKEN"
    }
    data = {
        "name": name
    }
    response = requests.post(url, json=data, headers=headers)
    return response.json()
# Usage Example
projects = get_projects()
if "desired_project_name" not in projects:
    create_project("desired_project_name")
```

Let's take a closer look at this code snippet:

1. We import the **requests** module to handle HTTP requests.
 2. The **get_projects** function sends a **GET** request to the Beagle Security API to retrieve existing projects associated with the provided access token.
 3. The **create_project** function sends a **POST** request to create a new project with the specified name.
 4. In this example, we fetch the existing projects and create a new one if the desired project name is not found in the list of projects retrieved.
2. **Create a new application:** Once the project framework has been constructed, we will proceed to build a new application beneath it:

```
def create_application(project_id, name, url):
    # Establish a new application within the designated project
    url = "https://api.beaglesecurity.com/rest/v2/applications"
    headers = {
        "Content-Type": "application/json",
        "Authorization": "Bearer YOUR_ACCESS_TOKEN"
    }
    data = {
        "projectId": project_id,
        "name": name,
        "url": url
    }
    response = requests.post(url, json=data, headers=headers)
    return response.json()

# Usage Example
project_id = "your_project_id"
application_name = "Your Application"
application_url = "https://your-application-url.com"
application = create_application(project_id, application_name, application_url)
```

Let's take a closer look at this code:

1. The **create_application** function sends a **POST** request to create a new application under the specified project.
 2. It requires parameters such as **project_id**, **name**, and **url** for the new application.
 3. In the usage example, we provide the project ID, application name, and URL to create a new application.
3. **Verify the domain:** Before testing, domain ownership verification is required to guarantee proper ownership and authorization for security assessments:

```
def verify_domain(application_token):
    # Retrieve domain verification signature
    url = f"https://api.beaglesecurity.com/rest/v2/applications/signature?application_token={
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN"
    }
    response = requests.get(url, headers=headers)
    return response.json()

# Usage Example
application_token = "your_application_token"
domain_verification_signature = verify_domain(application_token)
13.
```

Let's take a look at this code example:

1. The **verify_domain** function sends a **GET** request to obtain the domain verification signature for the specified application token.

2. It constructs the URL dynamically using f-strings to include the application token in the request.
3. In the usage example, we provide the application token to retrieve the domain verification signature.
4. **Start the test:** After domain validation, we begin the security test for our application:

```
def start_test(application_token):
    # Commence the test for the specified application
    url = "https://api.beaglesecurity.com/rest/v2/test/start"
    headers = {
        "Content-Type": "application/json",
        "Authorization": "Bearer YOUR_ACCESS_TOKEN"
    }
    data = {
        "applicationToken": application_token
    }
    response = requests.post(url, json=data, headers=headers)
    return response.json()

# Usage Example
test_start_response = start_test(application_token)
```

Here's an explanation for this code:

1. The **start_test** function sends a **POST** request to initiate the security test for the specified application token.
2. It includes the application token in the request payload.
3. In the usage example, we pass the application token to start the test.

Now, let's merge all of these functions into a single script that we can use for our automated workflow:

```
import requests
import sys

# Define global variables
BEAGLE_API_BASE_URL = "https://api.beaglesecurity.com/rest/v2"
ACCESS_TOKEN = "YOUR_ACCESS_TOKEN"

def get_projects():
    # Retrieve projects from Beagle Security
    url = f"{BEAGLE_API_BASE_URL}/projects"
    headers = {"Authorization": f"Bearer {ACCESS_TOKEN}"}
    response = requests.get(url, headers=headers)
    return response.json()

def create_project(name):
    # Create a new project if it doesn't exist
    url = f"{BEAGLE_API_BASE_URL}/projects"
    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {ACCESS_TOKEN}",
    }
    data = {"name": name}
    response = requests.post(url, json=data, headers=headers)
    return response.json()

def create_application(project_id, name, url):
    # Create a new application under the specified project
    url = f"{BEAGLE_API_BASE_URL}/applications"
    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {ACCESS_TOKEN}",
    }
```

```

    }
    data = {"projectId": project_id, "name": name, "url": url}
    response = requests.post(url, json=data, headers=headers)
    return response.json()

def verify_domain(application_token):
    # Verify domain ownership for the application
    url = f"{BEAGLE_API_BASE_URL}/applications/signature?application_token={application_token}"
    headers = {"Authorization": f"Bearer {ACCESS_TOKEN}"}
    response = requests.get(url, headers=headers)
    return response.json()

def start_test(application_token):
    # Start a security test for the specified application
    url = f"{BEAGLE_API_BASE_URL}/test/start"
    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {ACCESS_TOKEN}",
    }
    data = {"applicationToken": application_token}
    response = requests.post(url, json=data, headers=headers)
    return response.json()

def send_results_to_webhook(application_token, result_token, webhook_url):
    # Get test result
    url = f"{BEAGLE_API_BASE_URL}/test/result?application_token={application_token}&result_token={result_token}"
    headers = {"Authorization": f"Bearer {ACCESS_TOKEN}"}
    response = requests.get(url, headers=headers)
    test_result = response.json()
    # Send result to webhook
    webhook_data = {
        "application_token": application_token,
        "result_token": result_token,
        "result": test_result,
    }
    webhook_response = requests.post(webhook_url, json=webhook_data)
    return webhook_response.status_code

def main():
    # Check if project name argument is provided
    if len(sys.argv) < 2:
        print("Usage: python script.py <project_name>")
        sys.exit(1)

    # Extract project name from command-line arguments
    project_name = sys.argv[1]

    # Example usage
    application_name = "Your Application"
    application_url = "https://your-application-url.com"
    webhook_url = "https://your-webhook-url.com"

    # Retrieve projects or create a new one
    projects = get_projects()
    project_id = projects.get(project_name)
    if not project_id:
        new_project = create_project(project_name)
        project_id = new_project["id"]

    # Create a new application under the project
    new_application = create_application(project_id, application_name, application_url)
    application_token = new_application["applicationToken"]

    # Verify domain ownership
    domain_verification_signature = verify_domain(application_token)

    # Start a security test
    test_start_response = start_test(application_token)
    result_token = test_start_response["resultToken"]

    # Send results to webhook
    webhook_status_code = send_results_to_webhook(application_token, result_token, webhook_url)

```

```
print(f"Webhook status code: {webhook_status_code}")
if __name__ == "__main__":
    main()
```

The `main()` function serves as the entry point of our Python script. It orchestrates the various steps involved in automating application testing using Beagle Security's API. Let's break down each part of the `main()` function in detail:

1. **Argument validation:** The function begins by checking if the user has provided the required command-line arguments. In this case, we expect at least two arguments: the script name and the project name. If fewer than two arguments are provided, the function prints a usage message and exits with an error code.
2. **Project name extraction:** If the correct number of arguments is provided, the script extracts the project name from the command-line arguments. This is done using `sys.argv[1]`, which retrieves the second command-line argument (the first argument is always the script's name).
3. **Additional variables are defined:** Next, we define additional variables, such as `application_name`, `application_url`, and `webhook_url`. These variables represent the name, URL, and webhook URL of the application being tested, respectively. These values are placeholders and should be replaced with actual values relevant to your application. The following code block is related to the previous three points and demonstrates their implementation in Python:

```
def main():
    # Check if project name argument is provided
    if len(sys.argv) < 2:
        print("Usage: python script.py <project_name>")
        sys.exit(1)
    # Extract project name from command-line arguments
    project_name = sys.argv[1]
    # Example usage
    application_name = "Your Application"
    application_url = "https://your-application-url.com"
    webhook_url = "https://your-webhook-url.com"
```

4. **Retrieve or create a project:** The script calls the `get_projects()` function to retrieve a list of existing projects from Beagle Security. It then attempts to find the project specified by the user. If the project does not exist (`project_id` is `None`), the script creates a new project using the `create_project()` function and assigns the obtained project ID to `project_id`:

```
# Retrieve projects or create a new one
projects = get_projects()
project_id = projects.get(project_name)
if not project_id:
    new_project = create_project(project_name)
    project_id = new_project["id"]
```

5. **Create the application:** Once we know the project exists, the script proceeds to create a new application under the specified project. It calls the `create_application()` function, passing the project ID, application name, and URL as arguments. The function returns a dictionary

containing information about the newly created application, from which we extract the application token (**applicationToken**):

```
# Create a new application under the project
new_application = create_application(project_id, application_name, application_url)
application_token = new_application["applicationToken"]
```

6. **Verify the domain:** The script verifies domain ownership for the newly created application by calling the **verify_domain()** function with the application token as an argument. This step ensures that the security tests are conducted by the rightful owner:

```
# Verify domain ownership
domain_verification_signature = verify_domain(application_token)
```

7. **Start the test:** With domain ownership verified, the script initiates a security test for the application by calling the **start_test()** function with the application token as an argument. It then extracts **result_token** from the response, which is needed to retrieve the test results later:

```
# Start a security test
test_start_response = start_test(application_token)
result_token = test_start_response["resultToken"]
```

8. **Send the results to a webhook:** Finally, the script sends the test results to a webhook URL by calling the **send_results_to_webhook()** function with the application token, result token, and webhook URL as arguments. It prints the status code of the webhook's response for verification purposes:

```
# Send results to webhook
webhook_status_code = send_results_to_webhook(application_token, result_token, webhook_url)
print(f"Webhook status code: {webhook_status_code}")
```

By using Beagle Security's API and Python, we built a fully automated flow that we will now put into our CI/CD flow while using GitHub Actions as our tool of choice.

GitHub Actions enables you to define workflows directly within your repository's code base, automating tasks such as building, testing, and deploying your applications.

So, let's create a GitHub Actions workflow so that we can start a test on code push to the repository:

1. **Create a workflow file:** Begin by creating a **.github/workflows** directory in your repository if it doesn't already exist. Inside this directory, create a YAML file where you will define your GitHub Actions workflow. You can name this file whatever you like – for example, **beagle_security_test.yml**.
2. **Define the workflow's steps:** Define the steps of your workflow within the YAML file. These steps will include tasks such as checking out the code, running tests, and interacting with Beagle Security's API:

```
name: Beagle Security Test
on:
  push:
    branches:
```

```

- main # Adjust branch name as needed
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v2
      - name: Set up Python
        uses: actions/setup-python@v2
        with:
          python-version: '3.x' # Specify Python version
      - name: Install dependencies
        run: pip install requests # Install requests library
      - name: Run Beagle Security tests
        run: python beagle_security_test.py argument_value

```

Now that the GitHub Actions workflow is set up, we can integrate automated testing into the workflow using Beagle Security.

We'll use a Python script (**beagle_security_test.py**) to interact with Beagle Security's API and automate the testing process. The Python script contains functions to interact with Beagle Security's API, including retrieving projects, creating applications, verifying domains, and starting tests.

Within the GitHub Actions workflow, add a step to execute the Python script, ensuring that the necessary dependencies (for example, the **requests** library) are installed:

```

steps:
  ...
  - name: Run Beagle Security tests
    run: python beagle_security_test.py

```

By integrating automated testing with Beagle Security into GitHub Actions, you can speed up the process of delivering high-quality, safe software while reducing manual effort and increasing overall efficiency.

While APIs are designed to be flexible and for custom use cases, Beagle Security also offers plugins for all CI/CD tools to speed up the process for you. You can find the full documentation at

<https://beaglesecurity.com/developer/devsecopsdoc>.

To summarize, in this section, we identified OWASP ZAP and Beagle Security as our automated DAST tools, and we built two security pipelines while utilizing Jenkins and GitHub Actions. We only covered basic routines here; however, we can modify them to meet our needs.

In the next section, we'll learn how to achieve resilience and reliability in our automated workflows.

Ensuring reliability and resilience in automated workflows

Reliability and resilience are fundamental aspects of any automated workflow, especially in the context of DevOps, where CI/CD pipelines are prevalent. In this section, we will delve into various strategies and best practices that you can use to ensure the reliability and resilience of your automated workflows.

Robust error-handling mechanisms

Error handling is crucial in automated workflows to gracefully manage unexpected failures and errors. The following are some robust error-handling mechanisms:

- **Exception handling:** Implement **try-except** blocks to catch and handle exceptions that may occur during script execution. This allows for graceful degradation and prevents the entire workflow from failing due to isolated errors.
- **Logging:** Incorporate logging mechanisms to record errors, warnings, and informational messages. Detailed logs facilitate troubleshooting and provide valuable insights into the execution flow of automated workflows.
- **Meaningful error messages:** Ensure that error messages are informative and actionable, providing relevant details about the nature of the error and potential resolution steps.

Implementing retry logic

Transient failures, such as network timeouts or temporary service disruptions, are common in distributed systems. Implementing retry logic helps mitigate the impact of such failures:

- **Exponential backoff:** Use exponential backoff strategies when retrying failed operations to prevent overwhelming the system with repeated requests. Gradually increasing the interval between retries reduces the likelihood of exacerbating the issue.
- **Retry limits and expiry:** Define sensible limits on the number of retries and the maximum duration for retry attempts. Excessive retries can prolong downtime and increase resource consumption, while indefinite retries may indicate a systemic issue that requires manual intervention.

Building idempotent operations

Designing idempotent operations ensures that repeated executions produce the same outcome, regardless of the previous state:

- **Idempotent scripts:** Structure scripts and workflows so that they're idempotent, meaning they can be safely rerun without causing unintended side effects or inconsistencies in the system's state. This is particularly important in scenarios where retries or re-executions are necessary.
- **Transactional integrity:** Group related operations into transactional units to maintain atomicity and ensure data integrity. If a transaction

fails midway, mechanisms should be in place to roll back or compensate for partial changes to avoid data corruption.

Automated testing and validation

Continuous testing is integral to verifying the reliability and correctness of automated workflows:

- **Test automation:** Integrate automated tests, including unit tests, integration tests, and end-to-end tests, into the CI/CD pipeline to validate changes and configurations. Automated testing ensures that new features or modifications do not introduce regressions or unexpected behavior.
- **Test environments:** Maintain separate environments for testing and validation, mirroring production as closely as possible. Automatically provisioning and tearing down test environments helps ensure consistency and reproducibility across tests.

Documentation and knowledge sharing

Comprehensive documentation and knowledge sharing promote understanding and collaboration among team members:

- **Documentation standards:** Document workflows, scripts, configurations, and dependencies thoroughly to aid in onboarding and troubleshooting. Include information on prerequisites, inputs, outputs, and expected behavior to facilitate usage and maintenance.
- **Knowledge-sharing culture:** Foster a culture of knowledge-sharing and collaboration within the team. Conduct regular code reviews, share best practices, and organize training sessions to disseminate knowledge and promote continuous improvement.

Security and access control

Ensuring the security of automated workflows involves safeguarding access to sensitive resources and data:

- **Access controls:** Implement robust access controls and authentication mechanisms to restrict access to critical resources. Use **Role-Based Access Control (RBAC)** to grant permissions based on user roles and responsibilities.
- **Secret management:** Securely manage credentials, API keys, and other sensitive information using dedicated secret management solutions. Avoid hardcoding secrets in scripts or configuration files and utilize encryption and secure storage options.

By incorporating these strategies and best practices into your automated workflows, you can enhance their reliability, resilience, and security, thereby enabling smoother and more efficient software delivery processes.

To illustrate the strategies and best practices discussed in the previous sections, let's enhance the Beagle Security automation code to ensure reli-

ability and resilience in the automated workflow. We'll implement error handling and recovery in the existing code and explain the changes in detail to help you better understand their practical application:

```
import requests
import sys
import time
# Define global variables
BEAGLE_API_BASE_URL = "https://api.beaglesecurity.com/rest/v2"
ACCESS_TOKEN = "YOUR_ACCESS_TOKEN"
# Define maximum retry attempts
MAX_RETRIES = 3
def get_projects():
    # Retrieve projects from Beagle Security
    url = f"{BEAGLE_API_BASE_URL}/projects"
    headers = {"Authorization": f"Bearer {ACCESS_TOKEN}"}
    # Implement retry logic for network issues
    retries = 0
    while retries < MAX_RETRIES:
        try:
            response = requests.get(url, headers=headers)
            response.raise_for_status() # Raise an exception for HTTP errors
            return response.json()
        except requests.exceptions.RequestException as e:
            print(f"Error fetching projects: {e}")
            retries += 1
            if retries < MAX_RETRIES:
                print("Retrying...")
                time.sleep(5) # Wait for 5 seconds before retrying
            else:
                print("Max retries reached. Exiting...")
                sys.exit(1)
def create_project(name):
    # Create a new project if it doesn't exist
    url = f"{BEAGLE_API_BASE_URL}/projects"
    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {ACCESS_TOKEN}",
    }
    data = {"name": name}
    # Implement error handling for API responses
    try:
        response = requests.post(url, json=data, headers=headers)
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        print(f"Error creating project: {e}")
        sys.exit(1)
# Similarly, implement error handling for other functions: create_application, verify_domain, sta
```

Let's take a closer look at this code:

- **Error handling mechanisms:** We added robust error handling using **try-except** blocks to catch exceptions and handle HTTP errors gracefully. This ensures that the script doesn't crash abruptly and provides meaningful error messages.
- **Retry logic:** We implemented retry logic for network-related issues, such as timeouts or intermittent connectivity problems. The script retries failed requests for a predefined number of attempts before exiting.

By incorporating error handling and retry mechanisms into the automated workflow, we ensure reliability and resilience, thereby minimizing the impact of failures and enhancing the overall robustness of the system. These practices enable smoother execution, effective troubleshooting, and continuous improvement of the automated processes.

Now, let's look at how to add logging as part of continuously improving our pipeline. We will work on the same code and learn how to make it work.

Implementing a logger for security pipelines

Continuously monitoring automated workflows is required to detect errors and performance issues early. In this section, we'll look at the necessity of implementing logging in our tools. Later, these logs can be utilized to monitor critical metrics, performance indicators, and system health in real time.

Let's implement a logger in our code:

```
# Import necessary libraries
import logging

# Configure logging
logging.basicConfig(filename='automation.log', level=logging.INFO)

def main():
    # Configure logging
    logger = logging.getLogger(__name__)
    # Example usage
    project_name = "Your Project"
    application_name = "Your Application"
    application_url = "https://your-application-url.com"
    webhook_url = "https://your-webhook-url.com"
    try:
        # Retrieve projects or create a new one
        projects = get_projects()
        project_id = projects.get(project_name)
        if not project_id:
            new_project = create_project(project_name)
            project_id = new_project["id"]
        # Create a new application under the project
        new_application = create_application(project_id, application_name, application_url)
        application_token = new_application["applicationToken"]
        # Verify domain ownership
        domain_verification_signature = verify_domain(application_token)
        # Start a security test
        test_start_response = start_test(application_token)
        result_token = test_start_response["resultToken"]
        # Send results to webhook
        webhook_status_code = send_results_to_webhook(application_token, result_token, webhook_url)
        logger.info(f"Webhook status code: {webhook_status_code}")
        except Exception as e:
            logger.error(f"An error occurred: {e}", exc_info=True)
    if __name__ == "__main__":
        main()
```

Let's take a closer look at this code:

- **Logging:** We introduced logging to record events, errors, and status information during the execution of the automated workflow. Logging ensures visibility into the workflow's behavior and aids in troubleshooting and analysis.
- **Error logging:** We configured the script so that it logs errors, including exceptions and traceback information, in the specified log file. This enables operators to identify and address issues effectively.
- **Centralized monitoring:** By centralizing logs in a dedicated log file (`automation.log`), operators can easily monitor the script's execution and identify any anomalies or failures.

In this section, we implemented a logger for just one function. However, you can implement this logger for all the functions in the program. Later, these centralized logs can be used for monitoring, a process we can undertake by using the monitoring tools that will be explained in detail in later chapters.

Summary

This chapter explored the use of Python and third-party tools to create automated security pipelines. We looked at how to use Python's adaptability and third-party tools to automate several elements of security testing, such as vulnerability scanning and penetration testing. We discussed how Python can be integrated with popular third-party security solutions such as OWASP ZAP and Beagle Security's API. We used various examples and code snippets to show how Python scripts may interact with these tools to automate processes such as vulnerability detection, compliance testing, and security assessment.

Furthermore, we went over best practices for creating resilient and dependable automated security pipelines. We investigated solutions for handling errors, logging, and monitoring to ensure the resilience of our automated workflows.

Overall, this chapter gave you a comprehensive understanding of how to leverage Python and third-party tools to automate and enhance your security processes while learning practical techniques for building and maintaining robust security pipelines, ensuring your applications are secure and resilient.

The next chapter acts as a continuation of this chapter. There, you'll learn how to design security automation tools that are aligned with your requirements.